

Name: Aditya Bhusawale

Div: B **Batch:** B2

PRN No: 123B1B094

Assignment No. 2

Title - Implement a Coordinator (Leader) Election model in a distributed system using the Bully Algorithm, simulating failure detection and dynamic re-election among multiple processes.

Objectives

- To understand the importance of coordinator (leader) election in distributed systems and its role in achieving reliable and fault-tolerant coordination.
- To implement the Bully Algorithm in Python using threading and queue-based message passing to represent multiple independent processes running concurrently.
- To simulate the failure of the current coordinator and observe how active processes detect the failure and start a new election process.
- To demonstrate the sequence of ELECTION, OK, and COORDINATOR messages exchanged among processes until a new coordinator is selected.
- To analyze and compare the Bully Algorithm and Ring Algorithm based on message complexity, network topology, election process, and fault tolerance.

Problem Statement

In a distributed computing environment, multiple independent processes often need a single coordinator to handle important tasks such as resource locking, task scheduling, and transaction management. If the current coordinator fails or becomes unreachable, the system may experience delays, inconsistency, or interruptions. Implement a Coordinator Election Algorithm using the Bully Algorithm that enables active processes to identify coordinator failure and automatically conduct an election to select a new, common leader without requiring manual intervention.

Outcomes

- We successfully model a group of distributed processes with an active coordinator managing the cluster.
- We identify the failure of the existing coordinator and initiate a new election through a lower-ID process.
- We illustrate the flow of ELECTION, OK, and COORDINATOR messages, resulting in the highest-ID active process taking leadership.
- We ensure that all active processes eventually agree on a single newly elected coordinator.
- We develop practical understanding of thread-based concurrency, queue-based communication, and inter-process message exchange in distributed systems.

Requirements

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4 GB minimum
- Storage: 500 MB free disk space
- Network: Not required (nodes are simulated locally via threads)

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python 3.x
- IDE: Visual Studio Code / PyCharm
- Libraries: threading, queue, time
- Concepts: Distributed Systems, Coordinator (Leader) Election

Theory

1. Introduction

The Bully Algorithm is a well-known coordinator election technique used in distributed systems, introduced by Hector Garcia-Molina in 1982. It allows a group of independent processes in a fully connected network to automatically select a coordinator when the current coordinator fails or becomes unavailable. The algorithm is called the “Bully Algorithm” because the active process with the highest process ID takes priority over all lower-ID processes and eventually becomes the new coordinator.

1. Assumptions of the Bully Algorithm

- Each process P_i has a unique and totally ordered Process ID (PID), such as $P_1, P_2, \dots, P_n$.
- Every process maintains information about the IDs and communication addresses of the other processes.
- Communication between functioning processes is assumed to be reliable, with failures detected using predefined timeout limits.
- Any process, including the current coordinator, may fail during system operation.

2. Types of Messages

- ELECTION: Sent by a process to all active processes having a higher PID to start an election.
- OK / ANSWER: Returned by a higher-ID process to confirm that it is active and will participate in the election.
- COORDINATOR / VICTORY: Sent by the winning process to lower-ID processes to announce that it has become the new coordinator.

3. Operational Workflow

1. Failure Detection: A process P_i detects that the current coordinator is not responding, usually through a heartbeat failure or request timeout.
2. Election Initiation: Process P_i sends an ELECTION message to all processes having a higher PID than itself.
3. Response Handling:
 - If P_i receives an OK response, it stops competing for leadership and waits for the higher-ID process to complete the election.

- If P_i receives no response within the specified timeout, it assumes that no higher-ID process is active and declares itself the coordinator.
 - The winning process then sends a COORDINATOR message to the lower-ID processes.
4. Cascading Election: When a higher-ID process P_k receives an ELECTION message, it sends an OK response to the sender and starts a new election involving processes with IDs greater than k .

4. Comparative Summary: Bully Algorithm vs. Ring Algorithm

Metric / Parameter	Bully Algorithm	Ring Algorithm
Network Topology	Fully connected network where each process knows the other processes	Logical ring where each process communicates mainly with its successor
Best-case Message Complexity	$O(N)$, when the process immediately below the coordinator detects failure	$O(N)$, requiring a complete traversal of the logical ring
Worst-case Message Complexity	$O(N^2)$, when a low-ID process starts the election	$O(N)$, generally requiring about $2N$ messages for election and coordinator announcement
Fault Resilience	Can handle process failures effectively but may generate significant message traffic	Requires ring reconstruction or bypassing failed nodes when an intermediate process crashes

Code-bully_electi

on.py:

```
import threading
import time
import queue
```

```
class Process(threading.Thread):
    def __init__(self, pid, all_pids, message_queues):
        super().__init__()
        self.pid = pid
        self.all_pids = sorted(all_pids)
        self.queues = message_queues
        self.is_alive = True
        self.is_coordinator = False
```

```

self.running = True
self.in_election = False

def log(self, text):
    print(f"[Process P{self.pid}] {text}")

def send_message(self, target_pid, msg_type, sender_pid):
    if target_pid in self.queues:
        self.queues[target_pid].put((msg_type, sender_pid))

def start_election(self):
    if not self.is_alive or self.in_election:
        return
    self.in_election = True
    self.log("Initiating Election! Broadcasting ELECTION message to higher IDs...")
    higher_pids = [p for p in self.all_pids if p > self.pid]
    for higher_pid in higher_pids:
        self.send_message(higher_pid, "ELECTION", self.pid)

    # Simulated timeout window to collect responses
    time.sleep(1.0)
    if not getattr(self, '_received_ok', False):
        self.declare_victory()
    else:
        self.log("Received OK from higher process. Stepping back.")
        self._received_ok = False
        self.in_election = False

def declare_victory(self):
    self.is_coordinator = True
    self.log("*** I AM THE NEW COORDINATOR ***")
    lower_pids = [p for p in self.all_pids if p < self.pid]
    for p in lower_pids:
        self.send_message(p, "COORDINATOR", self.pid)

def run(self):
    self._received_ok = False
    while self.running:
        if not self.is_alive:
            time.sleep(0.5)
            continue
        try:
            # Fetch message from incoming queue
            msg_type, sender_pid = self.queues[self.pid].get(timeout=0.5)

```

```

        if msg_type == "ELECTION":
            self.log(f"Received ELECTION message from P{sender_pid}.")
            self.send_message(sender_pid, "OK", self.pid)
            threading.Thread(target=self.start_election, daemon=True).start()
        elif msg_type == "OK":
            self.log(f"Received OK response from higher process P{sender_pid}.")
            self._received_ok = True
        elif msg_type == "COORDINATOR":
            self.is_coordinator = False
            self.log(f"Acknowledged Process P{sender_pid} as current COORDINATOR.")
    except queue.Empty:
        pass

```

```

def main():
    pids = [1, 2, 3, 4, 5]
    queues = {pid: queue.Queue() for pid in pids}
    processes = {}

    print("=====")
    print(" DISTRIBUTED SYSTEM BULLY ELECTION SIMULATION ")
    print("=====")
    print(f"Initializing process cluster with IDs: {pids}\n")

    for pid in pids:
        p = Process(pid, pids, queues)
        processes[pid] = p
        p.start()

    time.sleep(1.0)

    # Initially, set highest ID node as initial coordinator
    print("--- Initializing Baseline Leader ---")
    processes[max(pids)].declare_victory()
    time.sleep(1.5)

    # Step 1: Simulate Leader Failure/Crash
    leader_pid = max(pids)
    print(f"\n[EVENT] Crash simulated on active Coordinator Process P{leader_pid}!")
    processes[leader_pid].is_alive = False
    processes[leader_pid].is_coordinator = False
    time.sleep(1.0)

    # Step 2: A lower ID process detects failure and initiates election

```

```

detector_pid = 2
print(f"\n[EVENT] Process P{detector_pid} detects Coordinator unresponsiveness.")
threading.Thread(target=processes[detector_pid].start_election, daemon=True).start()

# Wait for election state convergence
time.sleep(4.0)

print("\n--- Simulation Complete: Final Cluster State ---")
for pid in sorted(pids):
    status = "CRASHED" if not processes[pid].is_alive else ("COORDINATOR" if
        processes[pid].is_coordinator else "ACTIVE NODE")
    print(f"Process P{pid}: {status}")

# Terminate threads
for p in processes.values():
    p.running = False

if __name__ == "__main__":
    main()

```

Output-

Console Output (Execution Trace):

```

vaibhav@DESKTOP-B4H418G MINGW64 ~/OneDrive/Desktop/DSL/bully_algorithm
$ python bully_election.py
=====
DISTRIBUTED SYSTEM BULLY ELECTION SIMULATION
=====
Initializing process cluster with IDs: [1, 2, 3, 4, 5]

--- Initializing Baseline Leader ---
[Process P5] ★★★ I AM THE NEW COORDINATOR ★★★
[Process P1] Acknowledged Process P5 as current COORDINATOR.
[Process P2] Acknowledged Process P5 as current COORDINATOR.
[Process P3] Acknowledged Process P5 as current COORDINATOR.
[Process P4] Acknowledged Process P5 as current COORDINATOR.

[EVENT] Crash simulated on active Coordinator Process P5!

[EVENT] Process P2 detects Coordinator unresponsiveness.
[Process P2] Initiating Election! Broadcasting ELECTION message to higher IDs...
[Process P3] Received ELECTION message from P2.
[Process P4] Received ELECTION message from P2.
[Process P2] Received OK response from higher process P3.
[Process P2] Received OK response from higher process P4.
[Process P3] Initiating Election! Broadcasting ELECTION message to higher IDs...
[Process P4] Initiating Election! Broadcasting ELECTION message to higher IDs...
[Process P4] Received ELECTION message from P3.
[Process P3] Received OK response from higher process P4.
[Process P2] Received OK from higher process. Stepping back.

```

```
[Process P2] Received OK from higher process. Stepping back.  
[Process P3] Received OK from higher process. Stepping back.  
[Process P4] ★★★ I AM THE NEW COORDINATOR ★★★  
[Process P1] Acknowledged Process P4 as current COORDINATOR.  
[Process P2] Acknowledged Process P4 as current COORDINATOR.  
[Process P3] Acknowledged Process P4 as current COORDINATOR.
```

```
--- Simulation Complete: Final Cluster State ---
```

```
Process P1: ACTIVE NODE  
Process P2: ACTIVE NODE  
Process P3: ACTIVE NODE  
Process P4: COORDINATOR  
Process P5: CRASHED
```

Conclusion-

The implementation of the Bully Algorithm provides a practical demonstration of Coordinator (Leader) Election in a distributed system using Python's threading and queue modules. The simulation illustrates how distributed processes identify the failure of the current coordinator, initiate elections among higher-ID processes, and ultimately select the highest-ID active process as the new coordinator.

Through this project, we gained practical knowledge of important distributed system concepts, including failure detection, leader election, concurrent thread execution, and message passing. The comparison with the Ring Algorithm also helps in understanding the differences in message complexity, network structure, and fault tolerance between the two election approaches.